**Architecture Decision Records (ADR)**

**AgriOps AI: System Architecture & Component 2 / Agent 2 Decisions**

IMPORTANT

**SE3090 Mandatory Compliance Note**:
According to Section 14.2 and Section 20 of the SE3090 Assignment Specification, submitting an **Architecture Decision Record (ADR)** is a **mandatory requirement**. The ADR serves as primary written evidence for **Learning Outcome 4 (LO4)** and is directly examined during the final viva evaluation.

**Executive Summary of Decisions**

| Decision ID | Decision Title | Selected Choice | Key Rationale |
|---|---|---|---|
| **ADR-001** | React State Management Strategy | **Redux Toolkit (RTK)** | Centralized state management for multi-step approval workflows, real-time task board updates, and global auth caching. |
| **ADR-002** | Flutter Mobile State & Camera Strategy | **flutter_bloc + image_picker** | Predictable state machine for offline task evidence uploads, camera access, and reactive status tracking. |
| **ADR-003** | Agentic AI Framework & Multimodal Architecture | **Custom Orchestrator + LangGraph with Vision LLM** | Enables strict 4-agent delegation, multimodal (image + text) diagnostic ingestion, and non-pathological safety guardrails for Agent 2. |
| **ADR-004** | AI Shared State & Audit Persistence Strategy | **PostgreSQL Normalized Audit Tables (EF Core)** | Ensures full observability (AIWorkflows, AgentExecutions, ValidationResults) without storing sensitive credentials or raw prompt dumps. |
| **ADR-005** | Task Lifecycle & Worker Assignment Architecture | **7-State Machine + Skill Certification Engine** | Replaces basic CRUD with mandatory multi-actor verification (Worker execution → Photo evidence → Manager approval). |
| **ADR-006** | Cloud Deployment & Gateway Routing | **ASP.NET Core Monolithic REST API Gateway** | Guarantees React and Flutter communicate exclusively via ASP.NET Core REST API, protecting internal AI services and credentials. |

**ADR-001: React Web Application State Management Strategy**

Status

**Accepted**

Context & Problem Statement

The React Web Dashboard manages complex, real-time administrative workflows including:

- Interactive Kanban task boards for **Component 2**.
- Human-in-the-loop approval queues for **Agent 2** crop analysis recommendations.
- Global user authentication and role permissions (Farm Manager vs. Agronomist vs. Admin).

We needed a predictable state-management pattern that handles async API status polling, optimistic UI updates, and clean state sharing across un-mounted dashboard routes.

Options Considered

1. **React Context API + useReducer**: Built-in, lightweight, but leads to frequent re-renders across large tree components and lacks dev-tools middleware.
2. **Zustand**: Fast and minimal, but less structured for large group team environments.
3. **Redux Toolkit (RTK) + RTK Query**: Standardized, explicit action/reducer structure, powerful Redux DevTools, built-in asynchronous state handling (createAsyncThunk), and caching.

Decision Outcome

**Chosen Option**: **Redux Toolkit (RTK)**.

Positive Consequences

- **Predictable State Transitions**: Slice reducers strictly manage task transitions (fetchTasks, assignWorker, verifyEvidence, approveAIRecommendation).
- **DevTools Auditability**: Extremely helpful during SE3090 viva demo to inspect state diffs live.
- **Role-Based Guards**: Global auth slice seamlessly protects management routes.

Negative Consequences / Mitigation

- *Boilerplate*: Solved by using RTK slice generators (createSlice).

**ADR-002: Flutter Mobile Application State Management & Device Feature**

Status

**Accepted**

Context & Problem Statement

The Flutter mobile application is used by Field Workers in outdoor farm environments with intermittent connectivity. It must support:

- Task list rendering with state transitions (Assigned → In Progress → Pending Verification).
- Camera device integration for snapping task completion evidence and crop symptom photos.
- Secure JWT token storage and background sync.

Options Considered

1. **SetState / Provider**: Simple, but quickly becomes unmaintainable for multi-screen forms and camera streams.
2. **Riverpod**: Flexible, but higher learning curve for group members.
3. **flutter_bloc (BLoC Pattern) + image_picker**: Enforces strict separation of UI and business logic via Events and States (TaskLoading, TaskSubmitted, CropPhotoCaptured).

Decision Outcome

**Chosen Option**: **flutter_bloc + image_picker hardware integration**.

Positive Consequences

- **SE3090 Device Feature Rule**: Fulfills the mandatory requirement for a meaningful mobile device feature (camera photo capture & multipart upload).
- **Testability**: BLoCs can be unit-tested without mounting UI widgets (bloc_test).
- **Clear State Machine**: Maps 1-to-1 with Component 2's task lifecycle states.

**ADR-003: Agentic AI Subsystem Framework & Multimodal Diagnostic Architecture**

Status

**Accepted**

Context & Problem Statement

The SE3090 specification mandates a **Level 4 — Full AI** multi-agent subsystem that:

- Is NOT a generic chatbot or single-prompt generator.
- Features at least **4 distinct specialized agents** (Planning, Crop Analysis, Weather, Safety/Validation).
- Incorporates human-in-the-loop approval gates.

For **Agent 2 (Crop Analysis Agent)**, the system must ingest multimodal field inputs (photos + observation notes) to assess plant health safely.

Options Considered

1. **Single LLM Prompt with Vision**: Easy to build, but fails the SE3090 requirement for distinct specialized multi-agent roles and deterministic validation.
2. **Generic Chatbot UI**: Explicitly forbidden by SE3090 rules ("Don't do a chatbot").
3. **LangGraph / Custom Orchestration with Multimodal Vision Engine & Non-Pathological Safety Guardrails**: Structured DAG (Directed Acyclic Graph) connecting specialized agents with strict input/output JSON schemas.

Decision Outcome

**Chosen Option**: **Custom Python / LangGraph Orchestrator integrated into ASP.NET Core**.

Safety & Defense Architecture (Viva Defense)

- Agent 2 is designed specifically as an **AI-Assisted Crop Health Stress Assessor**, NOT a medical disease diagnostic tool. It identifies stress markers (chlorosis, wilting, nitrogen deficiency) and passes suggestions to **Agent 4 (Validation)** before human approval on React.

- This design decision prevents hallucinated chemical prescriptions and makes the system 100% safe and defensible during viva evaluation.

**ADR-004: PostgreSQL Database Schema Strategy for Shared AI State & Audit Trails**

Status

**Accepted**

Context & Problem Statement

The assignment requires persisting AI workflow states, execution summaries, tool calls, and validation results in PostgreSQL using Entity Framework Core migrations. Secret credentials or hidden prompt dumps must not be persisted.

Decision Outcome

We implemented a normalized 5-table AI audit schema in PostgreSQL:

1. AIWorkflows: Tracks overall workflow instance, type, initiator, and status (Running, PendingApproval, Completed).
2. AgentExecutions: Logs individual agent step details (AgentName, ExecutionTimeMs, InputPayload, OutputPayload).
3. ToolCalls: Audit trail for third-party API / DB tool executions (ToolName, InputArguments, ToolResult).
4. ValidationResults: Hard-coded rule evaluation outputs emitted by Agent 4 (IsSuccessful, FailureReason).
5. Approvals: Human-in-the-loop gate records (ActionDescription, Status, ReviewedByUserId, Comments).

Benefits

- Full compliance with SE3090 observability guidelines.
- Enables live execution history rendering on both React and Flutter.

**ADR-005: Task Lifecycle State Machine & Skill-Based Worker Assignment**

Status

**Accepted**

Context & Problem Statement

Basic CRUD operations on tasks are insufficient for a 3rd-year SE3090 project. Component 2 requires structured business logic for worker assignment and verification.

Decision Outcome

We designed a **7-Step Operational Pipeline** with **Worker Skill Matching**:

- **Skill Certification Verification**: When assigning a worker to a task (e.g., Fertilization), the API cross-references WorkerSkills against TaskType requirements (ChemicalHandling).
- **Photographic Evidence Gate**: A worker cannot mark a task Completed. They must upload photo evidence via Flutter, moving the status to Pending Verification. Only a Farm Manager can verify and set the status to Completed.

**ADR-006: Backend Gateway Architecture & Cross-Platform Communication Rule**

Status

**Accepted**

Context & Problem Statement

The assignment enforces the **Mandatory Backend Rule**: React and Flutter must communicate ONLY with the C# ASP.NET Core Web API backend. If Python AI services are used, they must operate as internal services called exclusively by ASP.NET Core.

Decision Outcome

- **C# ASP.NET Core Web API** acts as the single authoritative public REST gateway.
- Authenticates clients via JWT tokens and Role-Based Access Control ([Authorize(Roles = "FarmManager,Worker")]).
- Internal HTTP/gRPC service bridges ASP.NET Core to the Python Agentic AI subsystem.
- Frontends never call AI models or external weather APIs directly.